Experiment 1

LO2: Use git effectively for version control source code managements and collaborative managements

Introduction to Git and GitHub

Global Username Configuration

Command Prompt

```
git config --global user.name "kushwaha-ayush"
```

- **Purpose:** Sets the global Git username configuration. This ensures that every future code snapshot (commit) created on this machine is officially signed and tracked under the author profile kushwaha-ayush.

Global Email Configuration

Command Prompt

```
git config --global user.email "ayushk0801@gmail.com"
```

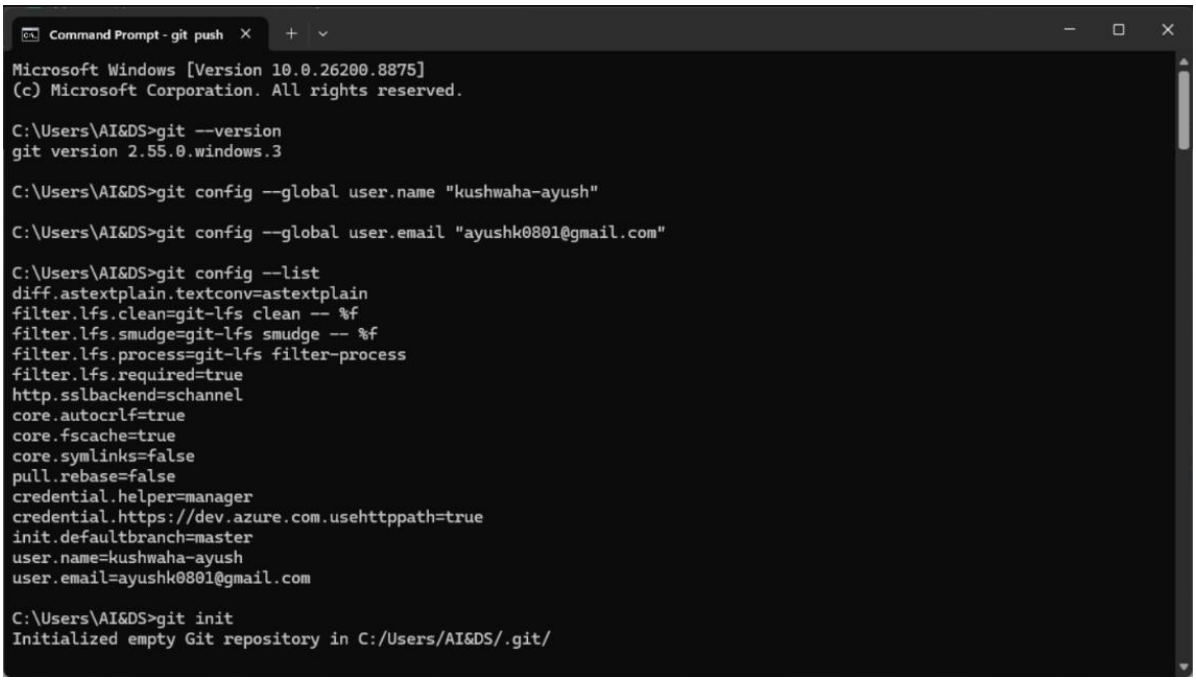
- **Purpose:** Sets the global Git email configuration to ayushk0801@gmail.com, ensuring that lab submissions and cloud platforms correctly identify the committer.

Listing Active Configurations

Command Prompt

```
git config --list
```

- **Purpose:** Displays the full active configuration profile of the local Git environment, confirming that user.name=kushwaha-ayush and user.email=ayushk0801@gmail.com were successfully saved, alongside the machine's default Git settings (credential helper, line-ending handling, default branch name, etc.).



Local Repository Initialization

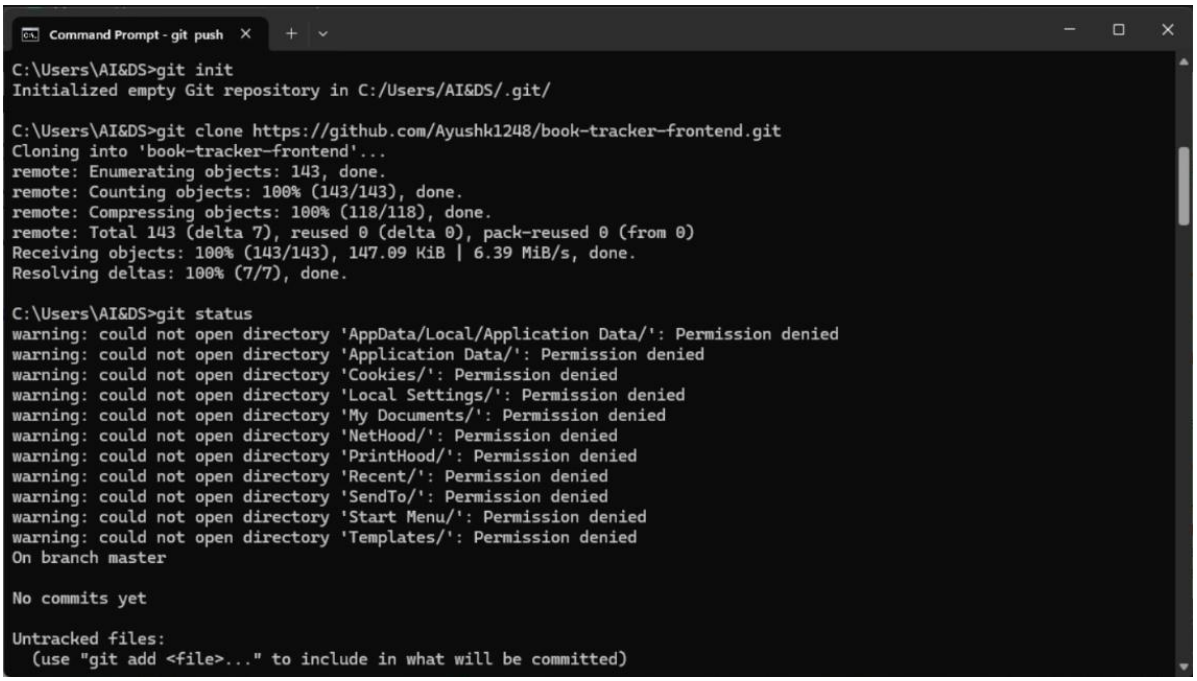
Command Prompt
`git init`

- **Purpose:** Initialized a brand new, empty local Git repository in the active directory path (`C:/Users/AI&DS/.git/`). This creates the hidden tracker system files required to begin version control on the local machine.

Successful Remote Repository Cloning

Command Prompt
`git clone https://github.com/Ayushk1248/book-tracker-frontend.git`

- **Purpose:** Downloads the target online repository directly onto the local workstation. The console reported 143 objects received and resolved, confirming the operation was successful and a clean workflow directory named `book-tracker-frontend` was linked to GitHub.

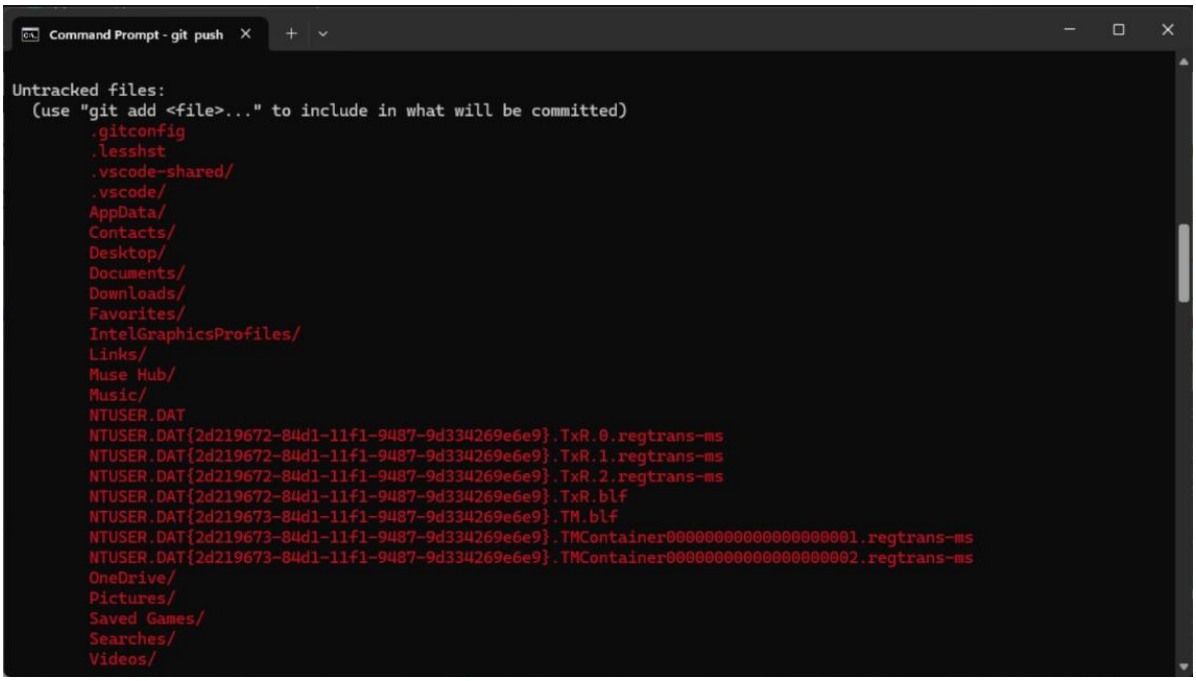


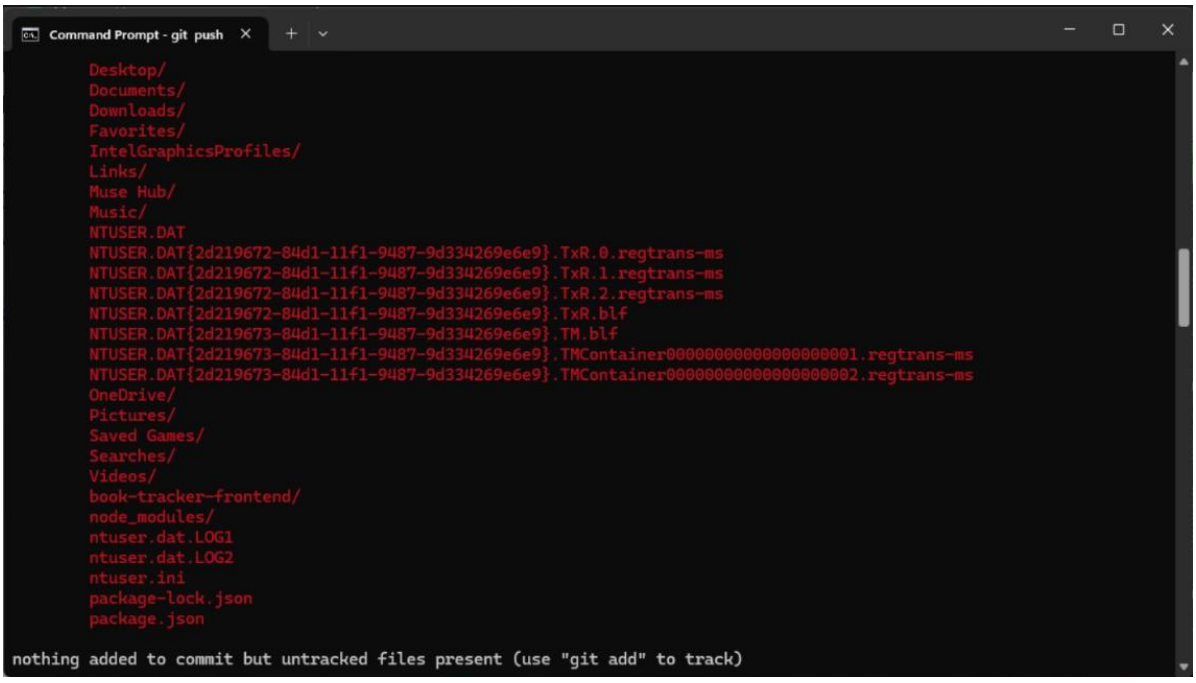
Repository Status Verification & Directory Scope Check

Command Prompt

`git status`

- **Purpose:** Checks the current state of the active repository. Because this was run inside the `User Home folder (C:\Users\AI&DS)` instead of inside the project subfolder, Git scanned the entire user directory.
    - **Permission Warnings:** The terminal generated multiple Permission denied warnings because Git attempted to scan protected Windows system directories (such as Application Data and Local Settings).
    - **Untracked Files (Red Output):** Git listed the entire contents of the user folder — including application settings, system caches, and the `book-tracker-frontend/` project folder — as untracked. This highlighted that the directory location needed to be changed before continuing.





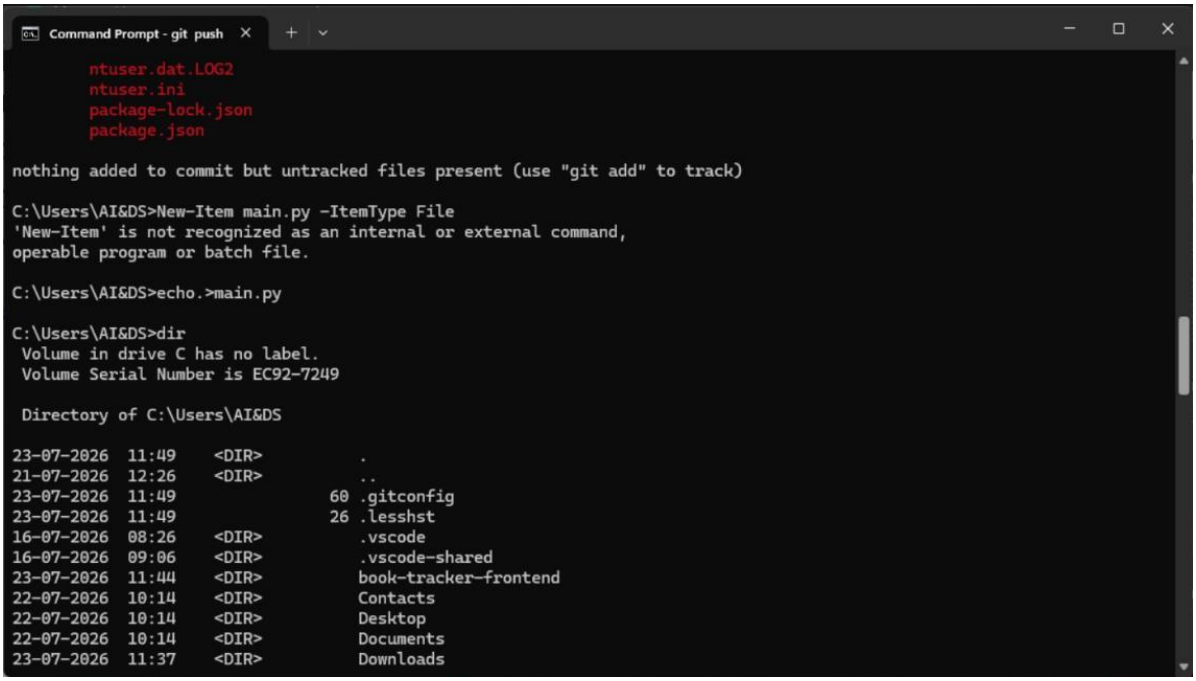
Empty Script File Creation

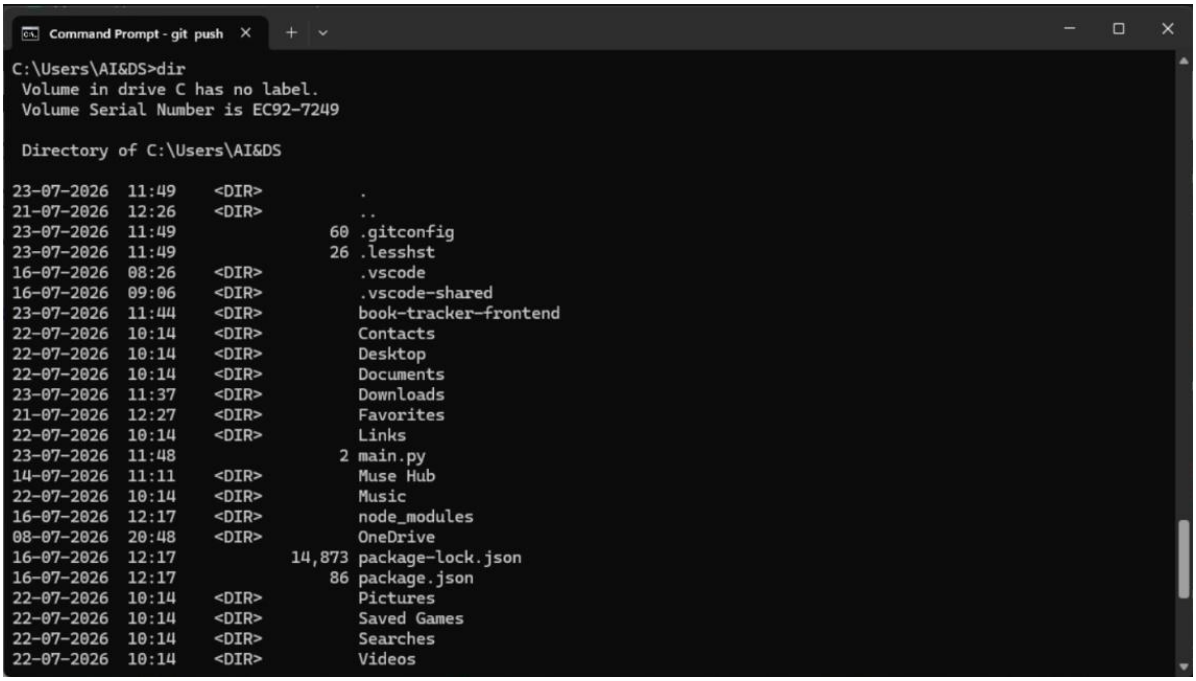
Command Prompt

```
New-Item main.py -ItemType File
```

```
echo.>main.py
```

- **Purpose:** The first attempt used `New-Item`, which is a PowerShell cmdlet and is not recognized inside Command Prompt, so it failed with an `'New-Item' is not recognized` error. The cmd-native equivalent, `echo.>main.py`, was used instead to create an empty `main.py` file, which was confirmed via a `dir` listing showing the new 2-byte file.





Project Content Staging & Commit Attempt

Command Prompt

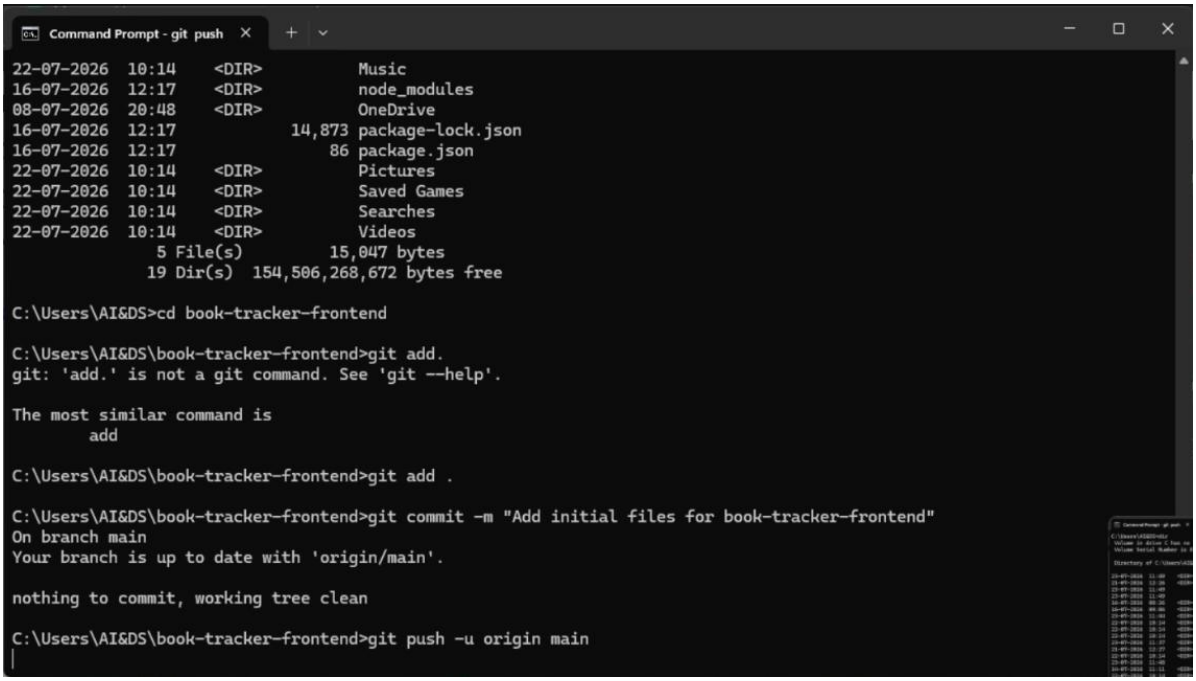
```
cd book-tracker-frontend

git add.

git add .

git commit -m "Add initial files for book-tracker-frontend"
```

- **Purpose:** Changed into the project directory and staged its contents. The first attempt, `git add.` (missing a space), was rejected as an invalid Git command; `git add .` staged the folder correctly. The subsequent commit reported `nothing to commit, working tree clean`, since `main.py` had actually been created one level up in the home directory rather than inside `book-tracker-frontend`, and the cloned repository already matched `origin/main`.

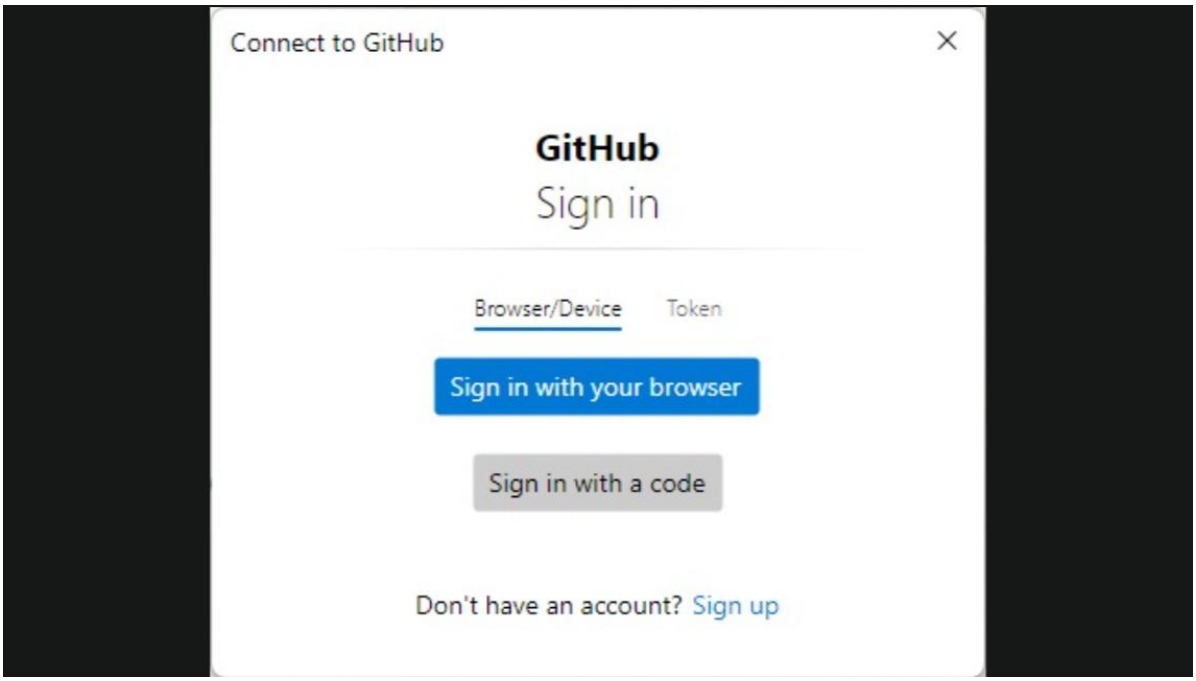


Remote Sync via GitHub Authentication

Command Prompt

```
git push -u origin main
```

- **Purpose:** Attempted to push the local branch upstream to `origin/main`. Since no valid cached credentials were found, Git's credential manager opened a `Connect to GitHub` prompt, requiring browser-based sign-in to authenticate before the push could complete.



Conclusion

This experiment demonstrated the foundational workflow of Git version control using Command Prompt, including configuring global user credentials, initializing and inspecting the local Git environment, and cloning a remote GitHub repository. It also highlighted a few common pitfalls encountered along the way — running status checks from the wrong directory, using a PowerShell-only command inside cmd, and a small typo while staging files — along with how each was diagnosed and corrected. The exercise concluded with staging and committing project files and authenticating with GitHub to sync the local repository with its remote origin, reinforcing the core add–commit–push cycle used in collaborative software development.